

Custom Shafts

Order Flow

How an order starts on **/dashboard/custom-shafts**, which function runs on each button, and where the order is actually saved.

ROUTE

/custom-shafts

PATHS

4 order starts

DATE

9 October 2026

OVERVIEW

Four ways an order starts

The catalog page does not save an order. It only chooses a path. The order is created on the next page.

PATH A

Catalog model

Konfigurieren on a product card.
`/custom-shafts/details/{modelId}`

PATH B

Own model

Eigenes Modell konfigurieren.
`/custom-shafts/product-order/new`

PATH C

Digital last

Leisten digital generieren.
`/leistenkonfigurator`

PATH D

Sole only

Bodenkonstruktion bestellen.
`/bodenkonstruktion`

Search, filters, favorites, the workflow graphic, **Demo ansehen**, and **Versandanleitung** do not create an order. Paths A and B share one start modal. After that, the buttons are the same. Chapter 6 records the only save-API difference.

Contents

1. What this page is
2. Catalog page before any order
3. Start modal for Path A and Path B
4. Path A — details page
5. Step 2 after Boden konfigurieren
6. Path B — own model
7. Path C — digital last
8. Path D — sole only
9. Where data sits between pages
10. End-to-end order map

What this page is

Route: `/dashboard/custom-shafts`

File: `app/(dashboard)/dashboard/custom-shafts/page.tsx`

The catalog lists maßschaft models. Choosing a model, an own model, a digital last, or a sole-only order leaves this page. Nothing is posted to the order API from the catalog itself.

Catalog page before any order

File: `app/(dashboard)/dashboard/custom-shafts/page.tsx`

2.1 Loading products

`useCustomShafts` calls `getAllCustomShafts`.

`GET /custom_shafts/mabschaft_kollektion`

Filters in state: `gender`, `category`, `sortOption`, `searchQuery`. Search waits 500 ms (`useDebounce`). **Mehr anzeigen** is `handleShowMore` and loads the next 8 products.

The favorite star is `handleToggleFavorite`. It updates the card first, then calls `toggleFavoriteModel`.

`POST /favorite_models/favorite` with `{ modelId }`. If the call fails, the star goes back.

`showPrices` (`useCustomShaftsShowPrices`) controls prices and whether the physical-last shipping wizard runs.

2.2 Values carried into the next page

Read from the URL: `orderId`, `customerId`, `customerName`, `feedbackId`, `sampleId`, `category`, `notificationId`, `modelId`.

`appendCustomShaftFlowParams` copies customer and Maßschuh linkage onto the next URL.

- `category` is `massschuhauftraege_order`, `LEISTENMASSCHAFT`, or `LEISTENKOMPLETTSCHUH`: the digital-last block and the sole-only footer are hidden.
- `LEISTENMASSCHAFT` or `LEISTENKOMPLETTSCHUH` plus `feedbackId` or `sampleId`: **Konfigurieren** skips the start modal and opens the details page with `source=3dupload`.
- `notificationId` and `modelId`: the matching product modal opens by itself.

Start modal used by Path A and Path B

File: `components/CustomShafts/NewSchaftErstellungModal.tsx`

First screen: SCHAFTERSTELLUNG

State `productionType` is `3d`, `physical`, or `draft`.

Choice	Next
3D-Modell hochladen	One step. <code>finishFlow</code> calls the 3D handler.
Physischer Leisten	Shipping wizard, then pickup or self-send. If prices are off, the wizard is skipped.
Als Entwurf fortfahren	Draft handler. No shipping.

Physical-last steps when prices are on:

- 1 Collective shipment (`shifting_bundle`).
- 2 Shipping label, only if extra orders were selected (`createLabel` is `yes` or `no`).
- 3 How the last is provided: self-send (`deliveryMethod = self`) or pickup (`pickup`).
- 4 Pickup only: shipping type, date, then address. Address is checked with `validateFedexShipping` .

The result object is `PhysicalLeistenSelection` . The page stores it and puts the fields on the next URL: `source=3dupload` , or `type=abholung` , or `is_draft=yes` , plus shipping, `lest_sending_type` , and `shifting_bundle` .

`lest_sending_type` is `feetfirst_shipping` for pickup and `self_shipping` for self-send.

Path	3D	Physical	Draft	Destination
A. Catalog	<code>handle3DUpload</code>	<code>handleAbholung</code>	<code>handleDraft</code>	<code>/custom-shafts/details/{id}</code>
B. Own model	<code>handleCustomOrder3DUpload</code>	<code>handleCustomOrderAbholung</code>	<code>handleCustomOrderDraft</code>	<code>/custom-shafts/product-order/new</code>

```
const finishFlow = (selectionOverride) => {
  const selection = selectionOverride ?? buildSelection();
  const mode3d = selectionOverride ? false : productionType === "3d";
  onPhysicalSelectionChange?.(selection);
  if (mode3d) onSelect3DUpload(selection);
  else onSelectAbholung(selection);
};
```

`components/CustomShafts/NewSchaftErstellungModal.tsx`

Catalog model on the details page

Route: `/dashboard/custom-shafts/details/{id}`

File: `app/(dashboard)/dashboard/custom-shafts/details/[id]/page.tsx`

The model id in the URL loads the shoe. The user edits it in `ProductConfiguration`. Main state: customer, left and right 3D files, pickup address, self-send data, leather, colors, shaft height, padding, zipper, closure, and notes.

3D upload fields are hidden when the start modal chose a physical last or a draft.

4.1 Weiter

The sticky bar calls `productConfigRef.submitOrder()`.

```
const submitOrder = () => {
  if (!runOrderValidation()) return;
  onOrderComplete();
};
```

`components/CustomShafts/ProductConfiguration.tsx`

If validation passes, `onOrderComplete` opens `ConfirmationModal`. A real 3D order also requires both last files. A draft does not.

4.2 Three buttons in the confirmation modal

File: `components/CustomShafts/ConfirmationModal.tsx`

Button	Function	Saved on this page?
Weiter ohne Boden	<code>handleOrderWithoutBoden</code>	Yes. Massschafterstellung only.
Boden konfigurieren	<code>handleBodenKonfigurieren</code>	No. Goes to Chapter 5.
Entwurf speichern	<code>handleSaveDraft</code>	Yes, only if <code>is_draft=yes</code> .

Both real buttons check the customer first. **Boden konfigurieren** also checks that a pickup bundle decision exists when that step is required.

4.3 Weiter ohne Boden — order created here

- 1 `handleOrderWithoutBoden` sets `pendingAction = 'ohne-boden'` and opens `ShaftPDFPopup`.
- 2 The PDF confirm opens the completion popup.
- 3 `proceedWithOrderWithoutBoden` builds the body and calls the API.
- 4 The app opens `/dashboard/balance-dashboard`.

Body builders: `prepareCollectionShaftData()`, then `prepareCollectionFormData()`. The catalog id is stored as `mabschaftKollektionId`. There is no custom shoe image.

Create. `createMassschuheWithoutOrderIdWithoutCustomModels`

POST `/custom shafts/create?custom_models=`

Existing draft. `updateAdminOrderAfterDraft` with `isCustomModels=false`

PATCH `/custom shafts/update-admin-order-after-draft/{orderId}`

`isCourierContact` is `no` when 3D files exist or the partner sends the last. It is `yes` when FeetF1rst picks the last up.

A new draft strips shipping fields and sends `is_draft=yes`, so the production basis can be chosen later.

4.4 Boden konfigurieren — handoff only

- 1 `handleBodenKonfigurieren` sets `pendingAction = 'boden'` and opens the shaft PDF.
- 2 On confirm, `prepareCollectionShaftData()` is stored with `setContextData`, including `shaftPdfBlob`, the product image, and `productId`.
- 3 `writeCustomShaftContextDraft` saves the same payload so a refresh on the next page can restore it.
- 4 Redirect to `/dashboard/massschuhauftraege-details/2` with product, customer, order, draft, and shipping params.

No create API runs in this step. The order is saved on step 2.

```
const handleBodenKonfigurieren = async () => {
  if (!isDraftFromQuery && !validateCustomer()) return;
  if (!isDraftFromQuery && ensurePickupBundleBeforeOrder("boden")) return;
  setPendingAction("boden");
  setShowPDFModal(true);
};
```

`app/(dashboard)/dashboard/custom-shafts/details/[id]/page.tsx`

Step 2 after Boden konfigurieren

Route: `/dashboard/massschuhauftraege-deatils/2`

Route file: `app/(dashboard)/dashboard/massschuhauftraege-deatils/[id]/page.tsx`

Form file: `.../Details/Bodenkonstruktion.tsx`

The route id `2` renders `Bodenkonstruktion`. This is the sole form for a shaft that was just configured. It is not the standalone sole page in Chapter 8.

5.1 Data from step 1

`useCustomShaftData()` provides `contextData`. On load, the form copies it into local state and writes `writeCustomShaftContextDraft`. The header uses the customer, product name, and shaft price from that object. The shaft price is added to the sole total (`grandTotal`).

5.2 What the user fills

Rahmen, Hinterkappe, Vorderkappe, Brandsohle, Sohlenversteifung, Sohlenaufbau, Absatzform, sole, thickness, and color.

State: `selected`, `optionInputs`, `textAreas`, `rahmen`, `selectedSole`, `sohlenaufbau`, `absatzFormConfigs`.

5.3 Weiter on this page

- 1 Sidebar **Weiter** calls `handleWeiterClick`.
- 2 `runFormValidation()` runs. Errors use `scrollToFirstBodenkonstruktionError`.
- 3 A valid form is stored with `writeBodenkonstruktionFormDraft`. `grandTotal` is saved in `localStorage` as `currentBalance`.
- 4 A real order opens `PDFPopup`. A draft opens the completion popup.
- 5 PDF confirm stores `pdfBlob` and opens `CompletionPopUp`.
- 6 Confirm calls `prepareFormDataForAdmin2`.
- 7 Success clears the shaft context and opens `/dashboard/balance-dashboard`.

```
const handleWeiterClick = async () => {
  const errors = runFormValidation();
  if (errors.length > 0) {
    setValidationErrors(errors);
    scrollToFirstBodenkonstruktionError(errors);
    return;
  }
  setShowModal(true);
};
```

Details/Bodenkonstruktion.tsx

5.4 Body and PDFs

`prepareFormDataForAdmin2` calls `prepareStep2FormData` in `utils/customShoeOrderHelpers`. It sends the shaft payload from Chapter 4 and the sole JSON from `prepareMassschafterstellungJson2WithSoleFields()`.

Form field	File	Where it comes from
invoice	shaft_invoice.pdf	Shaft PDF kept in step 1 context
invoice2	Boden PDF	PDF made on this page

`deliveryDate` and `delivery_length` come from the completion popup. `delivery_length` is the Komplettfertigung day count.

5.5 Which API saves the Komplettfertigung order

Situation	Function
New catalog model, no <code>orderId</code>	<code>createMasssschuheWithoutOrderIdWithoutCustomModels</code>
New own model (<code>uploadedImage</code> is set)	<code>createMasssschuheWithoutOrderId</code>
Existing order and shaft data is present	<code>sendMasssschuheOrderToAdmin2</code> or <code>sendMasssschuheCustomShaftOrderToAdmin2</code>
Existing order, shaft context missing	<code>handleFormSubmit</code> → <code>sendMasssschuheOrderToAdmin3</code>
Save draft	<code>persistBodenDraft</code> → <code>updateAdminOrderAfterDraft</code>
Turn a draft into a real order	<code>handleDraftFinalizeNeutralSubmit</code> → update, then <code>sendOrderToAdmin</code>

`sendOrderToAdmin` is `POST /v3/admin-order/manage/send-order-to-admin/{id}` .

`isCourierContact` is read from the shaft context with the same rule as Chapter 4.3.

Own model

Route: `/dashboard/custom-shafts/product-order/new`

File: `app/(dashboard)/dashboard/custom-shafts/product-order/[id]/page.tsx`

The header button **Eigenes Modell konfigurieren** opens the same modal from Chapter 3. There is no catalog model id.

The screen, **Weiter**, and the confirmation buttons are the same as Chapter 4. Only the payload and the create flag change.

	Catalog details	Own model
Shaft payload	<code>prepareCollectionShaftData</code>	<code>prepareCustomShaftData</code>
FormData	<code>prepareCollectionFormData</code>	<code>prepareStep1FormData</code>
Model marker	<code>mabschaftKollektionId</code>	uploaded image
Create API	<code>createMassschuheWithoutOrderIdWithoutCustomModels</code>	<code>createMassschuheWithoutOrderId</code>
Create URL	<code>POST /custom-shafts/create?custom_models=</code>	<code>POST /custom-shafts/create?custom_models=true</code>
Draft flag	<code>isCustomeModels=false</code>	<code>isCustomeModels=true</code>

Weiter ohne Boden stays on this page. `proceedWithOrderWithoutBoden` creates the order and opens `/dashboard/balance-dashboard`.

Boden konfigurieren stores `prepareCustomShaftData()` with `setContextData` (shaft PDF included) and opens the same step 2 from Chapter 5:

`/dashboard/massschuhauftraege-deatils/2?productId=new&...`

On step 2, `uploadedImage` marks the order as a custom model, so the save uses `createMassschuheWithoutOrderId` (`custom_models=true`). The PDF pair is still `invoice` = shaft and `invoice2` = sole.

If the URL already has an order id, it is sent as `shoe_order_id`. Finalizing a draft uses `handleDraftFinalizeNeutralSubmit`: update the draft, then `sendOrderToAdmin`.

Digital last

Start: `components/CustomShafts/LeistenDigitalGenerieren.tsx`

Button: Leisten digital generieren

This is not a shaft order. It starts a Halbprobe / Leistenerstellung order.

`useFeatureAccess` checks the feature path `/dashboard/digitale-leistenerstellung`.

- Feature is on: `handleGenerate` opens `/dashboard/leistenkonfigurator`.
- Feature is off: `LeistenkonfiguratorDeactiveModal`. No order.

Demo ansehen only plays a video. The workflow graphic under the button is not a link.

7.1 Leistenkonfigurator

File: `app/(dashboard)/dashboard/leistenkonfigurator/page.tsx`

The form is a last data sheet: header data, Sprengung, and Leistentyp. `handlePriceUpdate` recalculates the price when material or last type changes.

- 1 **Weiter** calls `handleContinue`, which runs `validateForm`.
- 2 A valid form opens the confirmation modal.
- 3 Confirm calls `handleConfirmationConfirm` and opens the PDF modal.
- 4 `handlePDFConfirm` stores the invoice blobs and uploads them with `uploadInvoiceBlobToS3`.
- 5 The completion popup calls `handleCompletionConfirm`.
- 6 `buildFormData` then `createLeistenkonfigurator`.

API. POST `/massschuhe-order/admin-order/halbprobenerstellung/create`

The body includes the 3D images, checklist JSON, price, and invoice PDFs.

After success, an order opened from an existing Maßschuh order returns to `/dashboard/massschuhauftraege/detail`. Every other finish opens `/dashboard/balance-dashboard`.

Sole only

Start: `components/CustomShafts/BottomFooter.tsx`

Button: Bodenkonstruktion bestellen

Page: `app/(dashboard)/dashboard/bodenkonstruktion/page.tsx`

This page is not Chapter 5. There is no shaft context and no shaft PDF. The partner already has a shaft and only orders the sole.

Control on the catalog	Result
Bodenkonstruktion bestellen	<code>/dashboard/bodenkonstruktion</code>
Als Entwurf fortfahren	Same page with <code>is_draft=yes</code> , plus the current customer and order query params
Versandanleitung	Address dialog only. No order.

8.1 Weiter on the sole-only page

- 1 The user fills the sole checklist and the customer name. State matches Chapter 5.2, but it lives on this page.
- 2 `handleWeiterClick` runs.
- 3 A real order must pass `runFormValidation`. Then the PDF modal opens.
- 4 A draft opens the save-or-bind popup immediately. Empty fields are allowed unless `highlight_missing=yes`.
- 5 Confirm uses `prepareCustomBodenkonstruktionFormData`.

```
const handleWeiterClick = async () => {
  if (isDraftFromQuery) {
    setShowModal2(true);
    return;
  }
  const errors = runFormValidation();
  if (errors.length > 0) {
    scrollToFirstBodenkonstruktionError(errors);
    return;
  }
  setShowModal(true);
};
```

`app/(dashboard)/dashboard/bodenkonstruktion/page.tsx`

Action	Function	API
New draft	<code>handleSaveDraft</code>	POST <code>/custom_shafts/custom-bodenkonstruktion/create?is_draft=yes</code>
Update an existing draft	<code>handleSaveDraft</code>	PATCH <code>/custom_shafts/update-admin-order-after-draft/{id}</code>
Bind a draft	<code>handleRequestBindOrder</code> , then <code>handleBindOrder</code>	update, then <code>sendOrderToAdmin</code> with <code>shifting_bundle=no</code>
New real order	<code>handleCreateOrder</code>	POST <code>/custom_shafts/custom-bodenkonstruktion/create</code>

`handleRequestBindOrder` uses `getBodenkonstruktionValidationErrors` and requires a customer name. Success opens `/dashboard/balance-dashboard` .

Where data sits between pages

Place	What it holds
URL query	Production choice, customer, parent order, draft flag, shipping
<code>sessionStorage</code> via <code>writeShiftingBundleToStorage</code>	Collective shipment and pickup address
React context <code>setContextData</code>	Shaft payload and shaft PDF while moving from Chapter 4 or 6 into Chapter 5
<code>writeCustomShaftContextDraft</code>	Same payload, used if step 2 is refreshed
<code>writeBodenkonstruktionFormDraft</code>	Sole form, written when Weiter is clicked on step 2
<code>localStorage</code> key <code>currentBalance</code>	Sole total shown in the balance step

End-to-end order map

```

/dashboard/custom-shafts
|
├─ A Konfigurieren
|   modal: 3D | physical last | draft
|   /custom-shafts/details/{id}
|   Weiter
|   └─ Weiter ohne Boden
|       POST /custom_shafts/create?custom_models=
|       └─ Boden konfigurieren
|           /massschuhauftraege-deatils/2
|           invoice = shaft PDF
|           invoice2 = sole PDF
|           → Komplettfertigung create or update
|
├─ B Eigenes Modell konfigurieren
|   /custom-shafts/product-order/new
|   └─ Weiter ohne Boden
|       POST /custom_shafts/create?custom_models=true
|       └─ Boden konfigurieren → same step 2
|
├─ C Leisten digital generieren
|   feature on → /leistenkonfigurator
|   POST ../halbprobenerstellung/create
|   feature off → no order
|
└─ D Bodenkonstruktion bestellen
    /bodenkonstruktion
    POST /custom_shafts/custom-bodenkonstruktion/create
    draft adds ?is_draft=yes
  
```

Finished orders open /dashboard/balance-dashboard.
 A digital last opened from an existing Maßschuh order
 returns to /dashboard/massschuhauftraege/detail.